`"(51.57, 0.17)"`. However, the coordinates are interchanged for the GeoJSON and WKT formats as longitude followed by latitude: for example, `"POINT (0.17 51.57)"`.

We can build various shapes using these formats. Table 12.1 provides a brief description of a few of them. See the Elasticsearch documentation on how to index and search documents to understand the concepts and examples in detail.

Table 12.1 Shapes supported by the `geo_shape` data type

Shape	Description	GeoJSON representation	WKT representation
Point	A point represented by latitude and longitude	`Point`	`POINT`
Multipoint	An array of points	`MultiPoint`	`MULTIPOINT`
Polygon	A multi-edge shape	`Polygon`	`POLYGON`
Multipolygon	A list of multiple polygons	`MultiPolygon`	`MULTIPOLYGON`
Line string	A line between two points	`LineString`	`LINESTRING`
Multiline string	A list of multiline strings	`MultiLineString`	`MULTILINESTRING`

Now that we know how to work with the indexing side of geodata using the `geo_point` and `geo_shape` fields, we're ready to search our documents. We discuss this in the next section.

12.3 Geospatial queries

We need to index London restaurant data for the examples in this section. The data is available in the datasets folder on GitHub (http://mng.bz/1q5R) and on the book's website.

To locate geospatial data, our next task is to search documents for a given geocriterion. For example, the place where you get your coffee is represented by a longitude and latitude, which, in turn, is called a *point* on the map.

As another example, I can search for the restaurants nearest my house, which returns each matching cafe represented as a point. Or a plot of land on a map can be represented by a shape representing a country or your local school's playground.

Elasticsearch provides a set of geospatial queries specifically suited to search for these use cases (such as finding nearby addresses or searching for all the interesting spots in a given area). We examine the following queries in the following sections:

- `geo_bounding_box` *query*—Finds documents enclosed in a rectangle constructed by geopoints; for example, all restaurants located inside a georectangle
- `geo_distance` *query*—Finds addresses within a certain distance from a point; for example, all ATMs within 1 km of London Bridge